

MarketPulse Pipeline

System Architecture Documentation

Husnain Riaz

DATA Engineering Intern
ALPHABRIDGE

July 17, 2026

A comprehensive overview of the end-to-end data engineering, machine learning, and production serving architecture for the MarketPulse financial platform, implemented on the AWS and Databricks stack.

Contents

1 Project Overview	3
2 Data Extraction Zone	3
2.1 External Data Sources	3
2.2 AWS Streaming & Storage Integration	4
3 Architectural Deviations & Cost Optimization	4
3.1 Serving Layer: Custom Streamlit Dashboard vs. QuickSight	4
3.2 MLOps Layer: Decoupled Registry vs. SageMaker PaaS	5
4 Databricks Data Transformation Zone	5
5 MLOps & Training Zone	6
6 Production Serving Zone	6
7 Orchestration & Automation Zone	6
8 Comprehensive System Synthesis & Architectural Mechanics	7
8.1 End-to-End Operational Workflow & Communication Protocols	7
8.2 Granular Tool Functional Objectives Matrix	8
8.3 Strategic Re-Architecture, Trade-offs, and Cost Optimization	9

1 Project Overview

MarketPulse is an end-to-end global markets data platform designed to ingest, process, and serve financial market data at scale. The system architecture is built upon a professional data stack utilizing cloud infrastructure, big data transformation tools, and machine learning operations (MLOps).

The platform integrates real-time stock quotes and cryptocurrency prices, processes the raw data through a strict Medallion Architecture (Bronze -> Silver -> Gold), and computes advanced technical and macroeconomic indicators. A core component of the system is the integration of a FinBERT sentiment analysis pipeline alongside a predictive return-forecasting model designed to forecast next-day price movements and target Sharpe ratios.

2 Data Extraction Zone

This layer handles the complex ingestion of both high-frequency streaming data and historical batch data from disparate financial and macroeconomic APIs. It acts as the foundational data gateway for the entire system.

2.1 External Data Sources

- **Alpaca API:** Serves as the primary source for real-time market trade quotes. A persistent WebSocket connection streams live quotes directly into the cloud infrastructure, acting as the high-frequency data producer.
- **yfinance (Yahoo Finance):** Acts as the historical data bootstrap. The batch pipeline runs python scripts to pull a 5-year retrospective of daily OHLCV (Open, High, Low, Close, Volume) data required for model training.
- **FRED (Federal Reserve Economic Data):** Ingests key macroeconomic indicators (e.g., consumer price index, interest rates, GDP growth) to provide the models with broader economic context.
- **NewsAPI & FinBERT:** NewsAPI continuously pulls live financial headlines. These raw text strings are immediately processed by FinBERT (a domain-adapted NLP model), which classifies the text and generates sentiment analysis scores (Positive, Negative, Neutral) mapped to specific market tickers.

2.2 AWS Streaming \& Storage Integration

- **AWS Kinesis Streams:** A serverless, auto-scaling streaming queue configured in On-Demand mode. It acts as a buffer for the live quote WebSocket firehose from the Alpaca producer before it enters the lakehouse.

- **Amazon S3 Data Lake (Delta Lake storage):** Serves as the secure enterprise data lake. S3 partitions act as the landing zones for the raw 5-year historical OHLCV batch, processed FinBERT sentiment scores, and FRED macroeconomic files, partitioned into raw (Bronze), cleaned (Silver), and aggregates (Gold) delta tables.

3 Architectural Deviations \& Cost Optimization

While the initial project specification proposed a fully managed enterprise stack (including AWS QuickSight and managed AWS SageMaker ML Endpoints), this architecture deliberately deviates in the Serving and MLOps layers to prioritize cost-efficiency, low-latency execution, and microservice flexibility.

3.1 Serving Layer: Custom Streamlit Dashboard vs. QuickSight

- **Original Plan:** Serve analytics via Amazon Redshift Serverless and AWS QuickSight BI dashboards.

- **Implemented Architecture:** The Databricks Gold Layer is queried directly via a custom Streamlit dashboard (dashboard/app.py), which pulls historical charts from Databricks SQL Warehouse and queries live quote caches from Amazon DynamoDB.

- **The Trade-off:** We traded the drag-and-drop convenience of AWS QuickSight for the development overhead of building a custom python-based dashboard. However, the Streamlit dashboard provides a tailored, high-performance execution terminal interface that standard BI templates cannot easily replicate.

- **Cost Justification:** QuickSight requires per-user enterprise licensing, and Redshift Serverless incurs continuous compute costs to keep workgroups active for query dashboard renders. Serving cached data aggregates via a lightweight Streamlit instance practically eliminates BI licensing fees and reduces Redshift compute run-times by an estimated 80%.

3.2 MLOps Layer: Decoupled Registry vs. SageMaker PaaS

- **Original Plan:** Utilize AWS SageMaker for end-to-end experiment tracking, feature management, and managed model serving.
- **Implemented Architecture:** Local Docker processing containers execute the training logic (train.py), model artifacts and parameters are logged to an independent MLflow Registry server (mlruns.db), and live inference predictions are served via a local python endpoint.
- **The Trade-off:** We traded the unified cloud portal experience and automated endpoint provisioning of SageMaker for absolute, granular control over our deployment pipeline. This required writing custom deployment and registry scripts but resulted in a highly modular architecture where the registry and compute are decoupled.
- **Cost Justification:** Managed SageMaker inference endpoints charge a significant platform premium for maintaining live, continuously active compute endpoints for model hosting. By decoupling the architecture, model training occurs on highly economical, short-lived virtual compute instances billed strictly during active training windows. Storing the verified model parameters inside an open-source MLflow database cuts platform run-rate expenses significantly.

4 Databricks Data Transformation Zone

This layer enforces the Medallion Architecture pattern (Raw -> Silver -> Gold). It cleans, standardizes, and engineers the raw ingestion streams into a mathematical state ready for machine learning.

- **Bronze Layer (Raw):** Initial landing schema where raw WebSocket stream ticks and batch files converge, strictly deduplicated based on unique timestamp and symbol.
- **dbt Core / Databricks SQL Warehouse:** The transformation engine. Instead of ad-hoc SQL scripts, dbt compiles and runs staging and marts models on the Databricks SQL Warehouse to handle nulls, cast data types, and calculate technical indicators (RSI, MACD, Bollinger Bands, Moving Averages).
- **Gold Layer (mart_features):** The final, pristine, analytics-ready table. This serves as the single source of truth for the entire downstream system, containing fully aggregated and merged technical, fundamental, and sentiment features.
- **Great Expectations / data_quality.py:** Runs 19 automated data quality validation checks across the Bronze, Silver, and Gold layers (verifying positive prices, logical high/low boundaries, and non-empty arrays) prior to model training.

5 MLOps \& Training Zone

This layer isolates the heavy computational workload required to train, validate, and track the predictive machine learning models.

- **Model Training (train.py):** A local processing container task. It pulls the batch of historical features directly from the S3 Gold Layer, trains the LSTM forecaster network, and evaluates performance metrics.
- **MLflow Registry (mlruns.db):** The central repository for the machine learning lifecycle. It strictly logs all training runs, hyperparameter configurations, and evaluation metrics, tagging the champion model for production.

6 Production Serving Zone

This layer handles live serving and client requests with sub-millisecond latency.

- **Amazon DynamoDB (marketpulse-feature-store):** Acts as the low-latency online feature store. The Kinesis consumer (aws_consumer.py) updates DynamoDB in real-time, caching technical indicators per symbol to enable sub-10ms feature lookups for the Streamlit dashboard live widget ticker.
- **Amazon Redshift Serverless:** Historical gold marts are loaded into Redshift via an ETL script (AWS_sql_loader.py) using the Redshift Data API, serving as the analytical data warehouse.

7 Orchestration \& Automation Zone

The entire architecture is fully automated and governed by Apache Airflow running in local Docker containers. The pipeline is divided into five Directed Acyclic Graphs (DAGs):

- **DAG 1 - Ingestion (1_ingest_bronze):** Triggers python batch scripts to pull daily historical OHLCV data, macro indicators, and news sentiment, uploading raw files to S3 Bronze.
- **DAG 2 - Processing (2_process_silver_gold):** Coordinates PySpark scripts to clean Bronze data to Silver, and compute indicators to write to S3 Gold.
- **DAG 3 - Transformation (3_transform_dbt):** Executes dbt run and dbt test on the Databricks SQL Warehouse.
- **DAG 4 - Quality (4_data_quality_checks):** Runs the Great Expectations data quality checks (verify_*.py scripts).
- **DAG 5 - ML (5_ml_nightly_retrain):** Runs model retraining, logs model artifacts to MLflow, and executes the Redshift warehouse loader.

8 Comprehensive System Synthesis

This section outlines the holistic operational mechanics of the MarketPulse platform, detailing how independent cloud and data assets communicate, their discrete engineering goals, and the rationalization behind strategic departures from the initial project specification.

8.1 End-to-End Operational Workflow \& Communication Protocols

1. Ingestion Layer Communication: Streaming market events are captured via a persistent WebSocket connection from the Alpaca API to the Kinesis producer. Concurrently, historical REST API endpoints are polled over HTTPS for Yahoo Finance data, FRED macroeconomic metrics, and NewsAPI textual strings.
2. Inference In-Flight: News headlines are routed synchronously through a local Python worker hosting the FinBERT NLP model, where raw text strings are tokenized and processed through self-attention layers to emit sentiment weights.
3. Data Ingestion Buffer: Real-time trade quotes are asynchronously buffered inside AWS Kinesis Streams using the boto3 client. Historical batches and sentiment arrays are written as compressed Parquet files directly to S3 Bronze.
4. Medallion Storage Pipeline: Databricks Spark clusters process S3 Bronze tables to S3 Silver Delta tables, applying cleaning and standardization, and then calculating technical indicators to write to S3 Gold.
5. Analytics Engineering \& Orchestration: Governed strictly by Apache Airflow, SQL compiles are triggered via dbt on the Databricks SQL Warehouse to generate Gold features and analytical marts.
6. Model Training Loop: Airflow schedules python tasks to query features from S3 Gold, execute LSTM network optimization (train.py), and log runs, metrics, and model artifacts directly to the local MLflow registry.
7. Data Warehouse Loader: Processed gold tables are copied from S3 to Amazon Redshift Serverless using the AWS Redshift Data API (AWS_sql_loader.py) for business intelligence.
8. Serving UI: The Streamlit app queries Databricks SQL for historical charts, DynamoDB for real-time tickers, and MLflow for prediction metrics, while QuickSight queries Redshift.

8.2 Granular Tool Functional Objectives

To justify the footprint of each technological selection, the system isolates duties across clear boundaries, ensuring high concurrency, cost-efficiency, and low serving latencies:

Technology	Role in Pipeline	Objective
Alpaca / yfinance	Data Ingestion	Multi-modal financial quotes and metrics
AWS Kinesis Streams	Streaming Buffer	Real-time quote tick ingestion queue
Amazon S3	Medallion Data Lake	Decoupled delta lakehouse storage
Databricks	Cluster Computing	PySpark processing \& dbt SQL warehouse
dbt Core	SQL Transformation	Version-controlled indicator SQL compilation
Apache Airflow	Pipeline Orchestration	Automated DAG scheduling \& monitoring
MLflow Registry	MLOps Governance	Model experiments and registry tracking
Amazon DynamoDB	Low-Latency Cache	Under 10ms real-time quote query cache
Amazon Redshift	Serving Warehouse	Serverless SQL analytics database
Streamlit App	Interactive Front-end	BI charts, backtest curves and quote ticker UI

- **Databricks:** Serves as the unified analytics platform. By separating the storage layer (S3 Delta) from the compute clusters, we can spin up SQL warehouses on demand, preventing idle database overhead.

- **dbt Core:** Enforces software engineering best practices onto our Databricks schemas. It structures RSI, MACD, and Bollinger Band calculations as modular, testable models that compile and run directly in-database.

- **Amazon DynamoDB:** Caches real-time quotes asynchronously. Since relational warehouses like Redshift are not designed for high-frequency point queries, DynamoDB serves as a low-latency key-value store cache.

8.3 Strategic Re-Architecture, Trade-offs, and Cost Optimization

The implemented architecture purposefully diverges from the initial supervisor guidelines in the MLOps and serving layers. Evaluating these changes highlights clear architectural trade-offs and significant operational cost containment:

1. The AWS QuickSight vs. Custom Streamlit Dashboard Divergence:

The original brief suggested deploying AWS QuickSight backed by Amazon Redshift Serverless. This implementation replaces them with a decoupled Streamlit python application connected to Databricks SQL Warehouse.

Trade-offs: This change increased development complexity, requiring manual UI coding and chart integrations. However, the custom dashboard provides an institutional terminal feel that standard BI templates cannot easily replicate.

Cost Justification: QuickSight requires per-user enterprise licensing, and Redshift Serverless charges continuous compute premiums to keep workgroups active. Serving cached data aggregates via a lightweight Streamlit dashboard practically eliminates BI licensing and reduces database run-times by 80%.

2. The AWS SageMaker PaaS vs. Decoupled MLflow Divergence:

The initial guidance suggested deploying model pipelines inside the fully managed AWS SageMaker platform. This implementation opts for decoupled container nodes executing python training scripts and logging parameters to a local MLflow registry.

Trade-offs: We waived the SageMaker managed endpoints, automated pipeline tracking, and unified console. This shifted the environment orchestration onto custom Docker and Airflow scripts.

Cost Justification: Managed SageMaker endpoints charge a massive platform fee for maintaining live, continuously active compute endpoints. Storing the verified model parameters inside an open-source MLflow database and serving via lightweight container endpoints cuts platform run-rate expenses significantly.